

Extending memory management tools

And so it is that we, as men, do not exist until we do; and then it is that we play with our world of existent things, and order and disorder them, and so it shall be that non-existence shall take us back from existence and that nameless spirituality shall return to Void, like a tired child home from a very wild circus.

- Omar Khayyam Ravenhurst and Malaclypse the Younger, Principia Discordia, pg 00058

I. Motivation

It is common for both the standard library and user libraries to manage memory without the use of standard-compliant allocators. They may use internal buffers (like `optional`) or use an allocator model that does not manage object lifetime [\[bde\]](#) [\[sgi\]](#) [\[eastl\]](#) [\[bitsquid\]](#). Such alternative allocator models are [common](#) in efficiency-critical applications.

Any container-like entity will need to solve the problems of how to create, move, copy, and destroy the objects which it maintains. Optimized, exception-correct implementations of these algorithms aren't necessarily obvious however. For example, `destroy` can be optimized into a no-op for trivially destructible types and `uninitialized_value_construct` can be optimized to `memset` for integral types. Having a standardized implementation of these features will help the performance and correctness of libraries.

II. Summary

The function templates `destroy_at`, `destroy` and `destroy_n` call the destructor for specified elements.

The function templates `uninitialized_move` and `uninitialized_move_n` perform move construction of elements over a range of memory, similar to `uninitialized_copy`.

The function templates `uninitialized_value_construct` and `uninitialized_value_construct_n` perform value-initialization of objects over a range of memory.

The function templates `uninitialized_default_construct` and `uninitialized_default_construct_n` perform default-initialization of objects over a range of memory.

Destruction order notwithstanding, these algorithms are isomorphic to existing features. The following table summarizes the relevant existing and proposed algorithms.

algorithm	feature
<code>uninitialized_value_construct</code>	array value-initialization
<code>uninitialized_default_construct</code>	array default-initialization
<code>uninitialized_copy</code>	<code>std::array</code> copy constructor
<code>uninitialized_move</code>	<code>std::array</code> move constructor
<code>uninitialized_fill</code>	(none?)
<code>destroy</code>	<code>~T[N]</code>
<code>destroy_at</code>	<code>~T</code>

Our wording for `destroy` depends on a fix for [LWG 2598](#).

This proposal directly contradicts [N0639](#) which removed earlier versions of these algorithms in 1995. That proposal removed `destroy(begin,end)` and `destroy(pointer)` which are identical to the `destroy` algorithms proposed here. That paper advocated removing these algorithms on the following grounds:

While they may be useful in implementing STL and collections similar to it, they are not really necessary for the purposes of the Standard Library, in that they do not aid communication between modules, implement semantics that cannot be reproduced by users, or provide substantial functionality. Furthermore, they are mostly subsumed by STL allocator members, which are what STL-conforming collections are expected to use.

It seems to this author fairly self-evident that this reasoning would not likely hold water if it were made today. Many important library features (like `move`) constitute simple but vital contributions to the vocabulary. Our analysis shows that this decision has forced user libraries and standard library implementations to re-invent the wheel.

III. Discussion

Organization

Each following section begins with a table. These tables represent an informal survey of existing practice, made mostly by searching github and the Visual Studio standard library implementation. They are by no means complete or scientific. Many standard library implementations already contain our algorithms, except in that they take an additional allocator parameter and forward memory management operations into that allocator object. This paper does not propose adding such algorithms. However, for completeness, those algorithms are included in the survey as "+ allocator"

In addition to these surveys, Creative Assembly has noted that they also have internal implementations of these algorithms.

destroy

P0040	destroy_at	destroy	+ allocator
VC++	_Destroy	~_Temp_iterator valarray::_Tidy	_Destroy_range
libstdcxx	_Destroy	_Destroy(first,last)	
libcxx		see valarray::resize destruct n:: process	vector:: destruct at end
EASTL	destruct	destruct(first,last)	
folly		see ~small_vector	S_destroy_range a
concrct		see concurrent_vector::_Destroy_array	
hpx		reinitializable_static::destruct	
bsl		ArrayDestructionPrimitives::destroy	

A design decision must be made regarding the order of destruction in `destroy`. Should bidirectional iterators destroy objects in reverse order? We argue no, for the following reasons:

- Order of construction and destruction is a property of the container, not a property of the iterator category. Not all containers with bidirectional iterators necessarily imply reverse-order destruction.
- It would be a very strange API which requires users to provide reverse iterators in order to destroy in forward order
- Reverse-order destruction can be provided by explicitly providing a reverse iterator (or, if truly desired, a `destroy_reverse` algorithm). Explicitness is the key here.
- The semantics of an algorithm should not change based on the iterator category. This could lead to supprises during code maintenance.
- Leaving the order as implementation-defined will likely damage its usefulness

It is conceivable that always destroying in forward order would be seen as suprising behavior to some. However, this is at least a consistent behavior which can be accounted for once understood. If we choose to change the behavior based on iterator category, we create a situation which is difficult for users to control.

A good implementation of `destroy` should not accept cases where expression `*first` resolves to an rvalue. For example, dereferencing an iterator could return `int` or `int&&` instead of `int&`. Returning an `int` is forbidden by our use of `ForwardIterator` but without Concepts we are as yet unable to enforce this, so an alternative error checking mechanism is needed. With appropriate resolution of [LWG 2598](#), the expression `addressof(*first)` will produce an error if `*first` returns an rvalue.

The specification of `destroy_at` may seem odd and restrictive in how it requires a non-null pointer rather than a reference or arbitrary iterator type. This design is chosen for the following reasons:

- It matches existing practice in various libraries (see table)
- It is conceptually compatible with `allocator::destroy` which operates on the allocator's pointer type
- By requiring a pointer, the API makes it difficult to call on a temporary object. If the algorithm operated on a reference or iterator, we would need to do extra work to exclude `const T&&s`
- The restrictive API makes it somewhat less likely to be called incorrectly. If the API accepted a reference, users might pass in a pointer and the pointer would be destroyed rather than the value to which it points.

All implementations of the standard library which were surveyed have some equivalent of the logic for `destroy` implemented in the exception handling block of the existing `uninitialized_*` algorithms.

The author wanted `destroy` to use `InputIterator` instead of `ForwardIterator`. As `destroy` is a single-pass algorithm, many of the features of `ForwardIterator` provide more guarantees than are strictly needed. Conversations on std-proposals and in private with Stepanov indicate broad disapproval for using `InputIterator` in this algorithm, so it has been restricted to the more conservative `ForwardIterator`. Objections revolved around questions regarding exception handling on iterator operations and a lack of compelling use cases.

uninitialized_move

P0040	uninitialized_move	+ allocator
-------	--------------------	-------------

VC++	see <code>vector::_Assign_rv (_Construct w/ move iterator)</code>	<code>_Uninitialized_move</code>
libstdcxx		_uninitialized_move_a
EASTL	uninitialized_move	
folly	moveToUninitialized	

Some concern is raised about exception handling with respect to `uninitialized_move`. If a move-constructor throws, the source object may have been irreparably damaged. As there is no solution to this problem, we implement the natural and expected semantics of destroying all fully constructed objects in the destination buffer and propagating the exception. This matches the behavior of `uninitialized_copy` as closely as possible.

An additional algorithm, `uninitialized_move_if_noexcept`, could be considered as a resolution to this concern. Such an algorithm was found already implemented in [libstdc++](#) using a `move_if_noexcept` iterator. Given that there is currently no range-based `move_if_noexcept` algorithm, such a solution is not considered here. It seems clear that such a feature is readily possible, however.

uninitialized_value_construct; uninitialized_default_construct

P0040	<code>uninitialized_value_construct</code>	+ allocator
VC++	<code>_Construct</code>	<code>_Uninitialized_default_fill_n</code> (deque, vector, internal_concurrent_hash)
libstdcxx	_uninitialized_default	<code>_uninitialized_default_a</code>
libcxx		vector::_construct at end
EASTL	uninitialized default fill	
folly	see Barrier::allocateControlBlock	
concrct	<code>_Internal_loop_guide::_Init</code>	
hpx	reinitializable_static::default_construct	

The names `uninitialized_value_construct` and `uninitialized_default_construct` were the originally proposed names, and were approved by ballot at Kona. Since that time, this author has grown to prefer the names `default_initialize` and `value_initialize` as they more clearly reflect the terminology used in the standard. The words "construct" and "initialize" already imply operating on uninitialized memory so the repetition seems redundant. In respect for Kona's decision, however, the names have been left as per their vote.

The table demonstrates that many libraries use the word "default" to refer to value-initialization. This is likely a historical artifact from a time before the distinction between default-initialization and value-initialization. It may also be caused simply by the fact that allocators provide no mechanism for containers to request one form of initialization over another. We seek to support multiple forms of initialization, so the distinction is important to make clearly.

The solution of using `uninitialized_fill` to fill a range with a default-initialized or value-initialized object is suboptimal, making this algorithm useful. Using fill presents the following issues:

- Requires unnecessary copy-construction which is not possible for move-only types
- May incur additional overhead

No algorithms equivalent to `uninitialized_default_construct` were found in our (very incomplete) search. The author believes that the Standard Library would be remiss to support only one form.

IV. Proposed Text

This document is written as a set of changes against n4582.

The existing structure of [specialized.algorithms] made amendments difficult, especially with respect to exception behavior. As such, the proposed wording applies a conservative refactoring in order to make the section more flexible. This wording is copied almost verbatim from 25.1 [algorithms.general]

A minor inconsistency was noted in that `uninitialized_copy` and `uninitialized_copy_n` are in the same section, but `uninitialized_fill` and `uninitialized_fill_n` are in separate sections. The fill algorithms are combined into one section as part of this wording.

Amend the <memory> header synopsis in 20.9.2 [memory.syn]

The header <memory> defines several types and function templates that describe properties of pointers and pointer-like types, manage memory for containers and other template types, [destroy objects](#), and construct multiple objects in uninitialized memory buffers (20.9.3–20.9.12).

Add the following to 20.9.2 [memory.syn]

```
template <class T>
    void destroy_at(T* location);

template <class ExecutionPolicy, class ForwardIterator>
    void destroy(ExecutionPolicy&& exec, //see [algorithms.parallel.overloads]
```

```
ForwardIterator first, ForwardIterator last);
```

```
template <class ForwardIterator>
void destroy(ForwardIterator first, ForwardIterator last);
```

```
template <class ExecutionPolicy, class ForwardIterator, class Size>
ForwardIterator destroy_n(ExecutionPolicy&& exec, //see [algorithms.parallel.overloads]
    ForwardIterator first, Size n);
```

```
template <class ForwardIterator, class Size>
ForwardIterator destroy_n(ForwardIterator first, Size n);
```

```
template <class ExecutionPolicy, class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_move(ExecutionPolicy&& exec, //see [algorithms.parallel.overloads]
    InputIterator first, InputIterator last, ForwardIterator result);
```

```
template <class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_move(InputIterator first, InputIterator last, ForwardIterator result);
```

```
template <class ExecutionPolicy, class InputIterator, class Size, class ForwardIterator>
pair<InputIterator,ForwardIterator> uninitialized_move_n(ExecutionPolicy&& exec, //see [algorithms.parallel.overloads]
    InputIterator first, Size n, ForwardIterator result);
```

```
template <class InputIterator, class Size, class ForwardIterator>
pair<InputIterator,ForwardIterator> uninitialized_move_n(InputIterator first, Size n, ForwardIterator result);
```

```
template <class ExecutionPolicy, class ForwardIterator>
void uninitialized_value_construct(ExecutionPolicy&& exec, //see [algorithms.parallel.overloads]
    ForwardIterator first, ForwardIterator last);
```

```
template <class ForwardIterator>
void uninitialized_value_construct(ForwardIterator first, ForwardIterator last);
```

```
template <class ExecutionPolicy, class ForwardIterator, class Size>
ForwardIterator uninitialized_value_construct_n(ExecutionPolicy&& exec, //see [algorithms.parallel.overloads]
    ForwardIterator first, Size n);
```

```
template <class ForwardIterator, class Size>
ForwardIterator uninitialized_value_construct_n(ForwardIterator first, Size n);
```

```
template <class ExecutionPolicy, class ForwardIterator>
void uninitialized_default_construct(ExecutionPolicy&& exec, //see [algorithms.parallel.overloads]
    ForwardIterator first, ForwardIterator last);
```

```
template <class ForwardIterator>
void uninitialized_default_construct(ForwardIterator first, ForwardIterator last);
```

```
template <class ExecutionPolicy, class ForwardIterator, class Size>
ForwardIterator uninitialized_default_construct_n(ExecutionPolicy&& exec, //see [algorithms.parallel.overloads]
    ForwardIterator first, Size n);
```

```
template <class ForwardIterator, class Size>
ForwardIterator uninitialized_default_construct_n(ForwardIterator first, Size n);
```

Amend [specialized.algorithms] in 20.9.12:

20.9.12 Specialized algorithms

[specialized.algorithms]

In the algorithm `uninitialized_copy`, the template parameter `InputIterator` is required to satisfy the requirements of an input iterator (24.2.3). In all of the following algorithms, the template parameter `ForwardIterator` is required to satisfy the requirements of a forward iterator (24.2.5), and is required to have the property that no exceptions are thrown from increment, assignment, comparison, or indirection through valid iterators.

Throughout this sub-clause, the names of template parameters are used to express type requirements.

- If an algorithm's template parameter is named `InputIterator`, the template argument shall satisfy the requirements of an input iterator ([input.iterators]).**
- If an algorithm's template parameter is named `ForwardIterator`, the template argument shall satisfy the requirements of a forward iterator ([forward.iterators]), and is required to have the property that no exceptions are thrown from increment, assignment, comparison, or indirection through valid iterators.**

Unless otherwise specified, in the following algorithms, if an exception is thrown **in the following algorithms** there are no effects.

Combine [uninitialized.fill] and [uninitialized.fill.n] in 20.9.12

20.9.12.4 `uninitialized_fill_n`

[uninitialized.fill.n]

Add the following to [specialized.algorithms] in 20.9.12:

20.9.12.4 `destroy`

[specialized.destroy]

```
template <class T>
```

```

void destroy_at(T* location);
    Effects: equivalent to:
        location->~T();

template <class ForwardIterator>
void destroy(ForwardIterator first, ForwardIterator last);
    Effects: equivalent to:
        for (; first!=last; ++first)
            destroy_at(addressof(*first));

template <class ForwardIterator, class Size>
ForwardIterator destroy_n(ForwardIterator first, Size n);
    Effects: equivalent to:
        for ( ; n > 0; (void)++first, --n)
            destroy_at(addressof(*first));
        return first;

```

20.9.12.5 uninitialized_move

[uninitialized.move]

```

template <class InputIterator, class ForwardIterator>
ForwardIterator uninitialized_move(InputIterator first, InputIterator last, ForwardIterator result);

```

Effects: equivalent to:

```

    for (; first != last; (void)++result, ++first)
        ::new (static_cast<void*>(addressof(*result)))
            typename iterator_traits<ForwardIterator>::value_type(std::move(*first));
    return result;

```

Remarks: If an exception is thrown, some objects in the range [first,last) are left in a valid but unspecified state.

```

template <class InputIterator, class Size, class ForwardIterator>
pair<InputIterator,ForwardIterator> uninitialized_move_n(InputIterator first, Size n, ForwardIterator result);

```

Effects: equivalent to:

```

    for ( ; n > 0; ++result, (void) ++first, --n) {
        ::new (static_cast<void*>(addressof(*result)))
            typename iterator_traits<ForwardIterator>::value_type(std::move(*first));
    }
    return {first,result};

```

Remarks: If an exception is thrown, some objects in the range [first,std::next(first,n)) are left in a valid but unspecified state.

20.9.12.6 uninitialized_value_construct

[uninitialized.construct.value]

```

template<class ForwardIterator>
void uninitialized_value_construct(ForwardIterator first, ForwardIterator last);

```

Effects: equivalent to:

```

    for (; first != last; ++first)
        ::new (static_cast<void*>(addressof(*first)))
            typename iterator_traits<ForwardIterator>::value_type();

```

```

template<class ForwardIterator, class Size>
ForwardIterator uninitialized_value_construct_n(ForwardIterator first, Size n);

```

Effects: equivalent to:

```

    for (; n>0; (void)++first, --n)
        ::new (static_cast<void*>(addressof(*first)))
            typename iterator_traits<ForwardIterator>::value_type();
    return first;

```

20.9.12.7 uninitialized_default_construct

[uninitialized.construct.default]

```

template<class ForwardIterator>
void uninitialized_default_construct(ForwardIterator first, ForwardIterator last);

```

Effects: equivalent to:

```

    for (; first != last; ++first)
        ::new (static_cast<void*>(addressof(*first)))
            typename iterator_traits<ForwardIterator>::value_type;

```

```

template<class ForwardIterator, class Size>
ForwardIterator uninitialized_default_construct_n(ForwardIterator first, Size n);

```

Effects: equivalent to:

```

    for (; n>0; (void)++first, --n)

```

```

::new (static_cast<void*>(addressof(*first)))
    typename iterator_traits<ForwardIterator>::value_type;
return first;

```

V. Revision history

R3

- Fixed various typos with ExecutionPolicy arguments and improved whitespace
- Converted section references to stable tags
- Effects are "equivalent to"
- No effects if an exception is thrown "unless otherwise specified"
- Altered editorial directives to reduce frequency of underlining
- Consistent (void) casting
- Changed "move" to "std::move"
- Fixed HTML formatting of iterator_traits<>
- Reworded uninitialized_move exception handling
- uninitialized_move_n returns a pair, removed references to LWG 2242
- Renamed begin/end parameters to first/last
- Consistent relative order of parallel vs. serial in synopsis
- Removed references to 25.1.12, using std::next instead
- Return statement refactor
- Throughout this [sub](#)-clause...

R2

- Added destroy_n, uninitialized_value_construct_n, uninitialized_default_construct_n for symmetry with existing algorithms.
- Tweaks to iterator categories and return values for better correctness
- Added destroy_at, which makes destroy both simpler and safer.
- Refactored [specialized.algorithm] for simpler wording
- Refactored discussions and surveys
- Rejected alternative wording for uninitialized_move
- Added references to LWG 2242 and 2598
- Added recommendation that we change the name of uninitialized_*_construct algorithms
- Made wording normative against n4582
- Reference and discussion of N0639
- Added cross-reference to 25.1.12
- Improved formatting
- Added amendments to memory.syn
- Added revision history

R1

- Added surveys of existing code
- Added discussion about order of destruction in destroy
- Added alternative wording for uninitialized_move
- Exempted destroy and uninitialized_move from no-effects policy

R0

- Initial revision

VI. Credits

Special thanks is given to Billy Baker and Patrice Roy for feedback and championship. Thanks to Michael Wong for helping to organize this effort. Thanks to Howard Hinnant for feedback regarding uninitialized_move. Thanks to Alexander Stepanov for consultations regarding destroy.